MACHINE LEARNING

(22PC1AM302) — B.Tech V Semester

LECTURE NOTES

UNIT II: Supervised Learning – I

Syllabus scope — Unit II: Linear and Non-Linear examples · Multi-Class & Multi-Label classification · Linear Regression · Multiple Linear Regression · Naïve Bayes Classifier · Decision Trees · ID3 · CART · Error bounds.

Recap — link from Unit I

Unit I framed learning as a search through a hypothesis space H, guided by training data, and asked two theoretical questions: which hypotheses are consistent with the data (Version Spaces) and how much data is "enough" to trust a hypothesis (PAC learning, VC dimension). Unit II turns to concrete supervised-learning models that realize this search in practice — for both classification (predicting a discrete label) and regression (predicting a continuous value) — and begins to ask how the error of these models should be measured and bounded.

Unit Learning Outcomes (mapped to CO-2)

- Distinguish linearly separable from non-linearly separable classification problems.
- Differentiate multi-class from multi-label classification and identify suitable algorithmic strategies for each.
- Fit and interpret Simple and Multiple Linear Regression models, including standard error/goodness-of-fit metrics.
- Apply the Naïve Bayes classifier to a labelled dataset using Bayes' theorem.
- Construct a decision tree using the ID3 algorithm (entropy and information gain) and describe the CART algorithm for classification and regression trees.
- Explain the idea of error bounds — training error vs. generalization error — and relate it back to the PAC/VC-dimension framework of Unit I.

1. Linear and Non-Linear Examples

Before studying individual algorithms, it is useful to classify classification problems by the shape of the boundary that separates their classes. This determines which family of models — linear or non-linear — is appropriate.

1.1 Linearly Separable Data

Definition: Linearly separable data

A binary classification dataset is linearly separable if there exists a straight line (in 2 dimensions), a plane (in 3 dimensions), or in general a hyperplane (in higher dimensions) that completely separates the two classes, with no misclassified points.

Linear models — such as linear regression, logistic regression, the single-layer perceptron and linear SVM — represent the decision boundary as a linear combination of the input features: $f(x) = w1x1 + w2x2 + \ldots + wnxn + b$. Because they have relatively few parameters, linear models are stable and have low variance, but higher bias — they underfit when the true boundary is curved.

Example: Linearly separable classification

Classifying emails using only the count of the word "free" and the count of exclamation marks may be roughly linearly separable: spam emails tend to have high values of both, while legitimate emails have low values, and a single straight line can approximately divide the two groups.

1.2 Non-Linearly Separable Data

Definition: Non-linearly separable data

A dataset is non-linearly separable if no single hyperplane can correctly separate the classes — the true decision boundary is curved or the classes are interleaved.

Such data cannot be captured well by a plain linear model; the classic example is the XOR problem, where the positive and negative classes lie on opposite diagonals and no straight line can separate them. Non-linear models handle this either by (a) using an inherently non-linear hypothesis — decision trees, k-NN, multi-layer neural networks —

or (b) transforming the input into a higher-dimensional feature space in which the classes become linearly separable, as is done by the kernel trick in non-linear SVMs.

> **Example: Non-linearly separable classification**
> The XOR function: inputs (0,0) and (1,1) belong to class 0, while (0,1) and (1,0) belong to class 1. Plotting these four points shows that any straight line will always misclassify at least one point — a curved or piecewise boundary is required.

1.3 Why This Distinction Matters

Aspect	Linear models	Non-linear models
Decision boundary	Straight line / hyperplane	Curved surface, or piecewise regions
Parameters	Few (one weight per feature)	Often more (tree splits, kernel/hidden units)
Bias / Variance	Higher bias, lower variance	Lower bias, higher variance (risk of overfitting)
Typical algorithms	Linear/Logistic Regression, Linear SVM, Perceptron	Decision Trees, k-NN, Kernel SVM, Neural Networks

Recognising whether a problem is (approximately) linearly separable is often the first practical step in choosing a model family, before moving on to specific algorithms such as those discussed in the rest of this unit.

2. Multi-Class and Multi-Label Classification

Real classification problems are not always binary (two-class). This section distinguishes the two common generalizations of binary classification.

2.1 Binary vs. Multi-Class Classification

Definition: Multi-class classification

A classification task with three or more mutually exclusive classes, where every instance is assigned to exactly one of the available class labels.

Binary classification (only two disjoint labels, e.g. spam / not-spam) is a special case with $|L| = 2$. Multi-class classification extends this to $|L| > 2$ disjoint labels — for example, classifying a handwritten digit as one of {0, 1, 2, …, 9}, or an iris flower as one of {Setosa, Versicolor, Virginica}.

Example: Multi-class classification

Handwritten digit recognition: each image belongs to exactly one of 10 classes (digits 0–9).

Iris flower species prediction: each flower belongs to exactly one of 3 species based on petal/sepal measurements.

Strategies for multi-class classification

- Native multi-class algorithms — decision trees, k-NN, Naïve Bayes and neural networks with a softmax output layer can directly handle more than two classes.
- One-vs-Rest (OvR / One-vs-All) — trains one binary classifier per class (that class vs. all others) and picks the class whose classifier is most confident.
- One-vs-One (OvO) — trains one binary classifier for every pair of classes and combines their votes; used, for example, to extend binary SVMs to multiple classes.
- Error-Correcting Output Codes (ECOC) — assigns each class a unique binary codeword and trains one classifier per bit of the code, giving redundancy that can correct some individual classifier errors.

2.2 Multi-Label Classification

Definition: Multi-label classification

A generalization of multi-class classification in which each instance may be assigned zero, one, or several non-exclusive labels simultaneously, rather than exactly one label from a disjoint set.

Formally, instead of predicting a single class y, a multi-label classifier predicts a binary vector y = (y1, y2, …, yk), where yi = 1 if label i applies to the instance and 0 otherwise. Binary and multi-class classification are both special cases of multi-label classification (with exactly one label always "on").

Example: Multi-label classification

Scene / image tagging: a single photograph might simultaneously be tagged "beach", "sunset" and "people" — all applicable at once.

Text categorization: a single news article may belong to both "Sports" and "Politics" categories if it discusses a government decision affecting a sporting event.

Common problem-transformation methods

- Binary Relevance — trains one independent binary classifier per label, predicting whether each label applies or not; simple, but ignores correlations between labels.
- Label Powerset (LP) — treats every distinct combination of labels seen in the training data as a single new class, converting the task into ordinary multi-class classification (with (2^k) possible classes for k labels).
- Classifier Chains — trains a sequence of binary classifiers, each additionally taking the predictions of previous classifiers in the chain as extra input features, so that label correlations can be captured.

2.3 Summary Comparison

Aspect	Binary	Multi-class	Multi-label
No. of labels	2	> 2	≥ 2 (non-exclusive)
Labels per instance	Exactly 1	Exactly 1	0, 1, or more
Example	Spam / not spam	Digit 0–9	Image tags: beach, sunset

3. Linear Regression

> **Definition: Linear Regression**
> *A supervised learning algorithm that models the relationship between a dependent (target) variable y and one or more independent (predictor) variables x by fitting a straight-line (linear) equation to the observed data.*

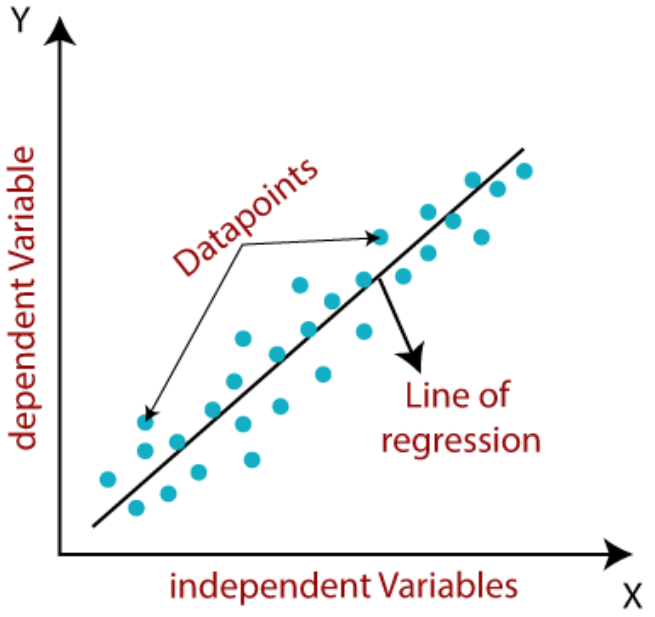
Linear regression answers the question "how can I use X to predict Y?" by drawing the best-fit straight line through the data — the line that lies as close as possible to all the observed points.

3.1 Simple Linear Regression

When there is a single independent variable, the model is called Simple Linear Regression, written as:

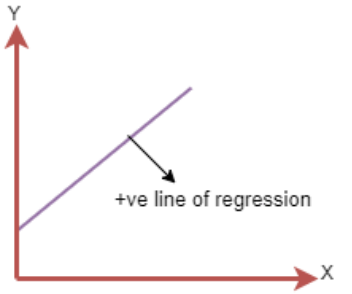
$y = a + bx + \varepsilon$

- y — the dependent / target variable (what we want to predict).
- x — the independent variable / feature (the known input).
- a — the intercept (predicted value of y when x = 0).
- b — the regression coefficient / slope (the change in y for a unit change in x).
- ε — the random error term, capturing the part of y not explained by x.

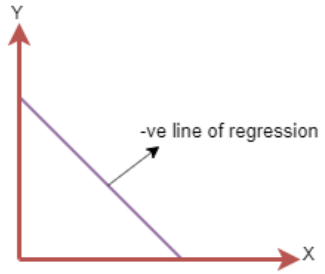


Linear regression fits a straight line through the scatter of data points, minimising the vertical distances (residuals) between the line and the points.

Sign of the slope



The line equation will be: $Y= a_0+a_1x$

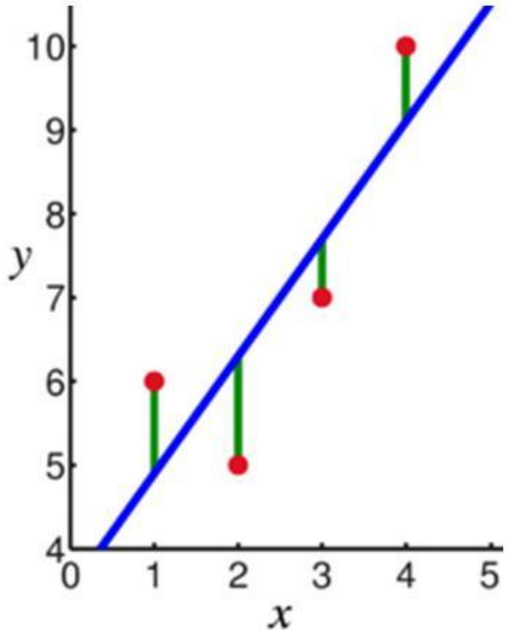


The line of equation will be: $Y= -a_0+a_1x$

A positive slope (b > 0) means y increases as x increases; a negative slope (b < 0) means y decreases as x increases.

Fitting the line — minimising squared error

The best-fit line is chosen to minimise the total squared vertical distance between each observed point and the line (its residual). For each training point, the residual is the difference between the observed value and the value predicted by the line:



Residuals (green lines) are the vertical gaps between each data point and the fitted line (blue).

This total is captured by the Mean Squared Error (MSE), the standard cost function minimised while fitting the line:

$$MSE = 1\frac{1}{N}\sum_{i=1}^{n}(y_i - (a_1x_i + a_0))^2$$

Smaller MSE indicates the line fits the training data more closely. The parameters a and b that minimise MSE can be found in closed form (the "normal equations" / ordinary least squares) or iteratively via gradient descent.

Goodness of fit — R-squared

Once fitted, the quality of the regression line is commonly reported using R^2 (R-squared), the proportion of the variance in y that is explained by x:

$$\text{R-squared} = \frac{\text{Explained variation}}{\text{Total Variation}}$$

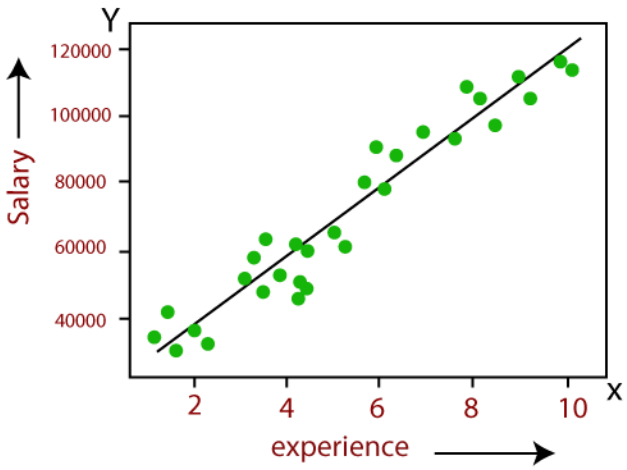
R^2 ranges from 0 (the model explains none of the variability) to 1 (the model explains all of the variability). Adjusted R^2 is a variant that penalises adding predictor variables that do not genuinely improve the fit, making it more appropriate when comparing models with different numbers of features.

Worked example

Consider advertisement spend ($) and the resulting sales ($) recorded over several months:

Advertisement	Sales
$90	$1000
$120	$1300
$150	$1800
$100	$1200
$130	$1380
$200	??

Fitting a simple linear regression line to this data allows us to predict sales for a new advertisement spend, e.g. estimating the expected sales when $200 is spent on advertising, by substituting x = 200 into the fitted equation y = a + bx.



Another example: Salary vs. Years of Experience — the fitted line lets us predict salary for a given number of years of experience.

4. Multiple Linear Regression

> **Definition: Multiple Linear Regression**
> *An extension of simple linear regression in which the dependent variable y is predicted using two or more independent variables x1, x2, ..., xn, assuming a linear relationship between each predictor and the target.*

The model equation becomes:

$$y = \beta 0 + \beta 1x1 + \beta 2x2 + \ldots + \beta nxn + \varepsilon$$

- y — the predicted value of the dependent variable.
- β0 — the intercept (value of y when every xi is 0).
- βi — the regression coefficient of predictor xi, representing its individual contribution to y holding all other predictors fixed.
- ε — the model (residual) error.

Multiple linear regression is useful whenever a target variable is more realistically explained by several factors together — for example, predicting a house's price from its area, number of bedrooms, age, and locality simultaneously, rather than from a single feature. Fitting is again done by minimising the sum of squared residuals (MSE) across all training examples, and the same R^2 / Adjusted R^2 metrics are used to judge the quality of fit — Adjusted R^2 being preferred here because plain R^2 can misleadingly increase merely by adding more predictors, whether or not they are actually useful.

Choosing between simple and multiple regression

Aspect	Simple Linear Regression	Multiple Linear Regression
No. of predictors	1	2 or more
Equation	y = a + bx	y = β0 + β1x1 + … + βnxn
Use case	Single dominant factor (e.g. experience → salary)	Several contributing factors (e.g. area, rooms, location → price)

4.1 Cost Function and Gradient Descent (brief note)

For both simple and multiple linear regression, the cost function quantifies the total error between the model's predictions and the actual values (typically the MSE). Gradient

Descent is the optimisation algorithm most commonly used to find the parameter values that minimise this cost: it iteratively adjusts each coefficient in the direction that reduces the cost the fastest (the negative gradient), taking small steps controlled by a learning rate until the cost converges to a minimum.

5. Naïve Bayes Classifier

Naive Bayes is a family of probabilistic classifiers built on Bayes' Theorem. Despite its simplicity, it remains a popular baseline algorithm because it is fast, easy to implement, and often performs surprisingly well.

Key Characteristics

- Simple yet powerful
- Fast training and prediction
- Works well with small datasets
- Handles multiple classes naturally
- Relies on a "naive" assumption of feature independence

Bayes' Theorem

$$P(y \mid x_1, x_2, \ldots, x_n) = \frac{P(x_1, x_2, \ldots, x_n \mid y) \cdot P(y)}{P(x_1, x_2, \ldots, x_n)}$$

- $P(y \mid x_1,\ldots,x_n)$ — posterior probability (what we want to find)
- $P(x_1,\ldots,x_n \mid y)$ — likelihood of the features given the class
- $P(y)$ — prior probability of the class
- $P(x_1,\ldots,x_n)$ — evidence (same for every class, so it can be ignored)

The Naive (Independence) Assumption

Naive Bayes assumes that all features are conditionally independent given the class label:

$$P(x_1, x_2, \ldots, x_n \mid y) = \prod_{i=1}^{n} P(x_i \mid y)$$

This assumption rarely holds perfectly in real data, but it dramatically simplifies computation and still produces good classification results in practice.

Classification Rule

$$\hat{y} = \operatorname{argmax}_y \; P(y) \prod_{i=1}^{n} P(x_i \mid y)$$

Steps for classification:

1. Calculate prior probabilities $P(y)$ for each class.
2. Calculate likelihoods $P(x_i \mid y)$ for each feature given each class.
3. For a new instance, compute the posterior score for each class.
4. Assign the class with the highest posterior score.

Note: The denominator $P(x_1,\ldots,x_n)$ is identical across all classes, so it can be dropped without affecting which class wins.

The Three Classic Variants Naive Bayes

- Gaussian NB — models each feature with a normal distribution (continuous data)
- Multinomial NB — models each feature as a count drawn from a multinomial distribution (discrete counts)
- Bernoulli NB — models each feature as a coin flip (binary presence/absence)

Gaussian Naive Bayes

Use case: continuous features that follow a (roughly) normal distribution, e.g., height, temperature, sensor readings.

Assumptions

- Features are continuous
- Each feature follows a Gaussian (normal) distribution within each class
- Features are conditionally independent

Likelihood Calculation

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_{y,i}^2}} \exp\left(-\frac{(x_i - \mu_{y,i})^2}{2\sigma_{y,i}^2}\right)$$

- μy,i — mean of feature i for class y
- σ^2y,i — variance of feature i for class y

Training Process

1. Calculate priors: P(y) = (number of samples in class y) / (total number of samples)

$$\mu_{y,i} = \frac{1}{N_y} \sum_{j\,:\,y_j = y} x_{j,i}$$

$$\sigma_{y,i}^2 = \frac{1}{N_y} \sum_{j\,:\,y_j = y} (x_{j,i} - \mu_{y,i})^2$$

Smoothing

To avoid a zero variance (which would make the formula blow up), add a small smoothing parameter ε:

$$\sigma_{y,i}^2 \leftarrow \sigma_{y,i}^2 + \epsilon$$

Worked Example — Classifying Gender from Height & Weight

Height (cm)	Weight (kg)	Gender
180	70	Male

170	60	Female
175	65	Male
165	55	Female
185	75	Male

- P(Male) = 3/5 = 0.6, P(Female) = 2/5 = 0.4
- Male: μheight = 180, σ²height = 25
- Female: μheight = 167.5, σ²height = 12.5
- (Weight statistics are computed the same way.)

Multinomial Naive Bayes

Use case: discrete features that represent counts or frequencies — most commonly word counts in text classification.

Common Applications

- Text classification (word counts)
- Document categorization
- Spam filtering

Feature Representation

Features represent counts: $x_i \in \{0, 1, 2, 3, ...\}$. The likelihood follows a multinomial distribution — it models how often each outcome (word) occurs.

Likelihood Calculation

$$P(x_i \mid y) = \frac{N_{y,i} + \alpha}{N_y + \alpha \cdot n}$$

- Ny,i — count of feature i in class y
- Ny — total count of all features in class y
- α — smoothing parameter (usually 1 → "Laplace smoothing")
- n — number of features

Worked Example — Email Spam Classification

Email	"free"	"money"	"meeting"	Class
1	2	1	0	Spam
2	0	0	3	Ham
3	1	2	0	Spam
4	0	0	1	Ham

- P(Spam) = 2/4 = 0.5, P(Ham) = 2/4 = 0.5
- Spam totals: N(spam,"free") = 3, N(spam) = 6
- P("free" | Spam) = (3+1)/(6+3·1) = 4/9

Laplace Smoothing

Adding α = 1 prevents zero probabilities for features (words) that were never observed in a class during training.

Bernoulli Naive Bayes

Use case: binary features that simply indicate presence or absence, e.g., "does the word appear at all?"

Feature Representation

Features are binary: $x_i \in \{0, 1\}$, where 1 = feature present and 0 = feature absent.

Likelihood Calculation

$$P(x_i \mid y) = \begin{cases} p_{y,i} & \text{if } x_i = 1 \\ 1 - p_{y,i} & \text{if } x_i = 0 \end{cases}$$

$$p_{y,i} = \frac{N_{y,i} + \alpha}{N_y + 2\alpha}$$

where Ny,i is the number of samples in class y in which feature i is present.

Worked Example — Document Classification (Binary Features)

Document	"urgent"	"meeting"	"sale"	Class
1	1	0	1	Spam
2	0	1	0	Ham
3	1	0	1	Spam
4	0	1	0	Ham
5	1	1	0	Ham

- P(Spam) = 2/5, P(Ham) = 3/5
- p(spam, "urgent") = (2+1)/(2+2) = 0.75
- p(ham, "urgent") = (1+1)/(3+2) = 0.4

Extra Tip: Bernoulli ≠ "Multinomial with 0/1 counts"

The fundamental Naive Bayes assumption

- **Feature independence:** This means that when we are trying to classify something, we assume that each feature (or piece of information) in the data does not affect any other feature.
- **Continuous features are normally distributed:** If a feature is continuous, then it is assumed to be normally distributed within each class.

- **Discrete features have multinomial distributions:** If a feature is discrete, then it is assumed to have a multinomial distribution within each class.
- **Features are equally important:** All features are assumed to contribute equally to the prediction of the class label.
- **No missing data:** The data should not contain any missing values.

Applications

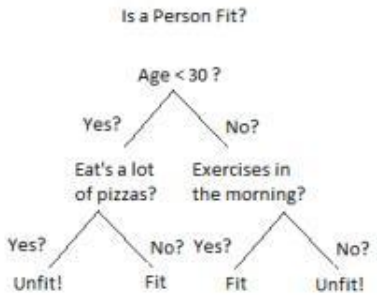
- **Spam Email Filtering**: Classifies emails as spam or non-spam based on features.
- **Text Classification**: Used in sentiment analysis, document categorization and topic classification.
- **Medical Diagnosis:** Helps in predicting the likelihood of a disease based on symptoms.
- **Credit Scoring:** Evaluates creditworthiness of individuals for loan approval.
- **Weather Prediction**: Classifies weather conditions based on various factors.
-

6. Decision Trees

> **Definition: Decision Tree**
>
> *A supervised learning model that predicts the value of a target variable by recursively splitting the data according to decision rules inferred from the features, represented as a tree of decision nodes (splits) and leaves (final outcomes).*

A decision tree has two kinds of elements: decision nodes, where the data is split on the value of some feature, and leaves, which hold the final predicted class (or value).
Decision trees are of two main kinds:

- Classification trees — the target/outcome is categorical (Yes/No, or one of several classes).
- Regression trees — the target/outcome is a continuous numeric value.



A simple classification tree predicting fitness from Age, diet and exercise habits.

6.1 The ID3 Algorithm

ID3 (Iterative Dichotomiser 3) is one of the earliest and most widely taught algorithms for building a classification tree. It builds the tree top-down, greedily choosing at each step the attribute that best separates the classes.

Entropy

> **Definition: Entropy H(S)**
> *A measure (in bits) of the amount of uncertainty or randomness in a set S of labelled examples; entropy is 0 when all examples belong to a single class (no uncertainty) and is maximal when the classes are evenly mixed.*

$$H(S) = \sum_{x \in X} p(x) \log_2 \frac{1}{p(x)}$$

Intuitively, a fair coin toss (P(heads) = P(tails) = 0.5) has maximum entropy, since the outcome is completely unpredictable; a two-headed coin has zero entropy, since the outcome is always known in advance.

Information Gain

> **Definition: Information Gain IG(S, A)**
> *The reduction in entropy of set S that results from partitioning S according to the values of attribute A; it measures how much "information" about the class label is gained by knowing A.*

IG(S, A) = H(S) − Σx P(x) · H(x), where x ranges over the values taken by attribute A, and H(x) is the entropy of the subset of S having that value.

ID3 proceeds recursively:

1. Create a root node for the (remaining) data.
2. If all examples belong to one class, return a single leaf node with that class.
3. Otherwise, compute H(S), the entropy of the current node's data.
4. For each remaining attribute, compute the information gain IG(S, A) that would result from splitting on it.

5. Select the attribute with the maximum information gain as the split for this node, and remove it from further consideration below this branch.
6. Recurse on each resulting subset until every branch ends in a leaf, or attributes are exhausted.

Worked Example — the classic "Play Golf" dataset

Consider 14 days of weather records (Outlook, Temperature, Humidity, Wind) and whether golf was played, with 9 "Yes" and 5 "No" outcomes overall.

Step 1 — Entropy of the whole dataset S:

$$Entropy(S) = \sum_{x \in X} p(x) \log_2 \frac{1}{p(x)}$$

$$Entropy(S) = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right)$$

$$= 0.940$$

Step 2 — to evaluate splitting on the attribute Wind (values Weak / Strong), first find the proportion of examples having each value:

$$P(S_{weak}) = \frac{Number\ of\ Weak}{Total}$$

$$= \frac{8}{14}$$

$$P(S_{strong}) = \frac{Number\ of\ Strong}{Total}$$

$$= \frac{6}{14}$$

Step 3 — compute the entropy within each subset:

$$Entropy(S_{weak}) = -\left(\frac{6}{8}\right)\log_2\left(\frac{6}{8}\right) - \left(\frac{2}{8}\right)\log_2\left(\frac{2}{8}\right)$$

$$= 0.811$$

$$Entropy(S_{strong}) = -\left(\frac{3}{6}\right)\log_2\left(\frac{3}{6}\right) - \left(\frac{3}{6}\right)\log_2\left(\frac{3}{6}\right)$$

$$= 1.000$$

Step 4 — combine these into the information gain for Wind:

$$IG(S, Wind) = H(S) - \sum_{i=0}^{n} P(x) * H(x)$$

$$IG(S, Wind) = H(S) - P(S_{weak}) * H(S_{weak}) - P(S_{strong}) * H(S_{strong})$$

$$= 0.940 - \left(\frac{8}{14}\right)(0.811) - \left(\frac{6}{14}\right)(1.00)$$

$$= 0.048$$

Repeating this calculation for every attribute gives:

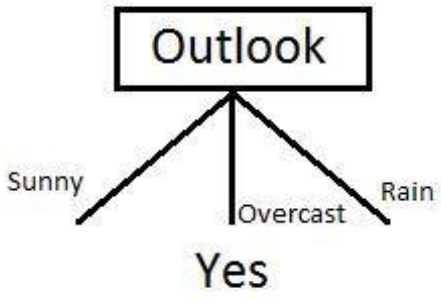
$IG(S, Outlook) = 0.246$

$IG(S, Temperature) = 0.029$

$IG(S, Humidity) = 0.151$

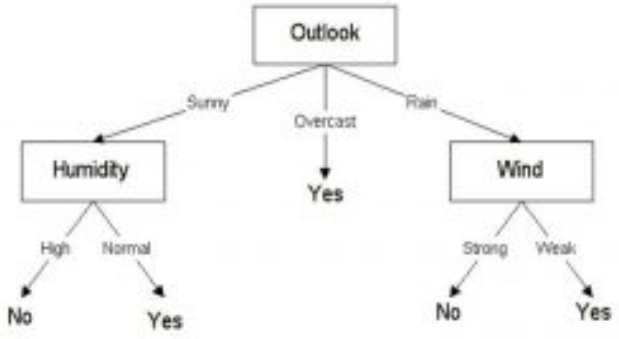
$IG(S, Wind) = 0.048$ (Previous example)

Since Outlook has the highest information gain (0.246), it is chosen as the root split of the tree.



Outlook becomes the root node, branching into Sunny / Overcast / Rain.

The algorithm then recurses within each branch (e.g. splitting the "Sunny" subset further on Humidity, and the "Rain" subset on Wind), eventually producing the complete decision tree:

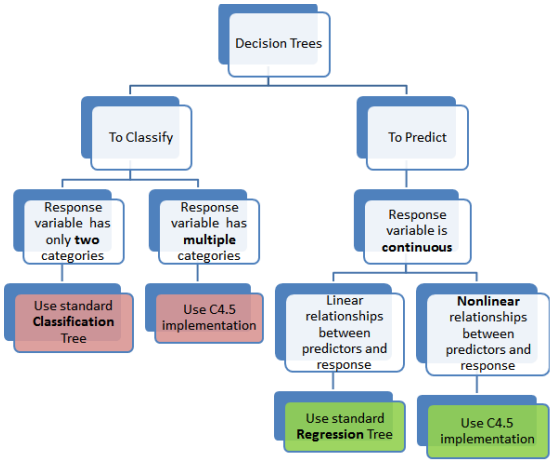


The final ID3 decision tree for the Play-Golf dataset.

7. Classification and Regression Trees (CART)

Definition: CART

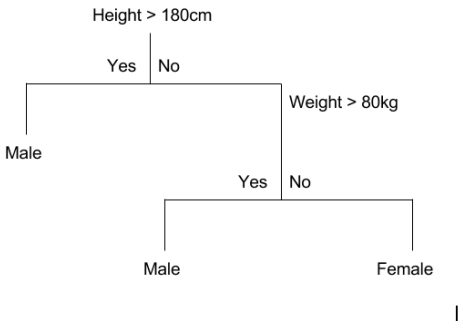
A decision-tree algorithm that can build both classification trees (categorical target) and regression trees (continuous target), where every internal node is a binary split on a single predictor variable and every leaf holds a prediction.

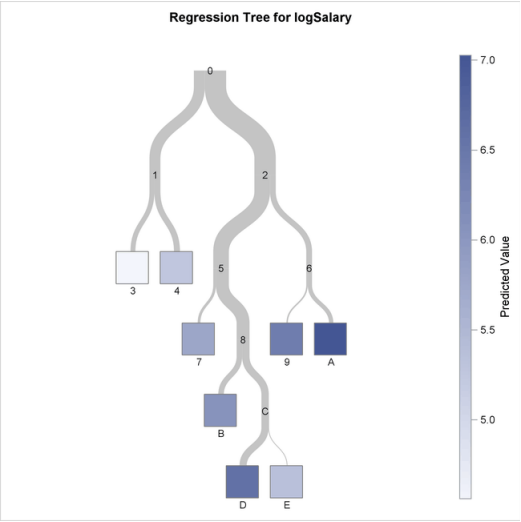


CART covers both classification trees (standard, or C4.5 for >2 categories) and regression trees, depending on the nature of the response variable.

7.1 Classification Trees vs. Regression Trees

Classification trees split the data to maximise the "purity" of the resulting groups, using impurity measures such as entropy (as in ID3) or the Gini index. Regression trees, in contrast, are used when the target is continuous (e.g. price, temperature): at each candidate split point, the Sum of Squared Errors (SSE) between the predicted and actual values is computed, and the split (variable and threshold) that gives the lowest SSE is chosen. This process is applied recursively to build the full tree.





7.2 Building a CART Tree — Steps

1. Greedy recursive binary splitting — the input space is divided by trying many possible split points across all variables and choosing the one that optimises a cost function (impurity for classification, SSE for regression).
2. Stopping criterion — splitting halts once a leaf's data falls below a minimum size threshold (or another user-set limit), which then becomes a leaf node.
3. Tree pruning — since large trees tend to overfit, branches that do not meaningfully improve performance on a hold-out validation set are pruned back, producing a simpler and more general tree.

Splitting Criteria in CART

CART needs a way to measure 'how good' a split is. It uses different formulas depending on the task.

A. Splitting Criteria for Classification — Gini Impurity

- Gini Impurity measures how mixed up the classes are in a node
- It tells us: if we randomly picked a sample and randomly labeled it based on the class distribution, how likely is it to be wrong?

$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

Where:

- $\mathbf{p_i}$ = proportion of class i in the node
- **n** = number of classes

- **Gini = 0** → node is completely pure (only one class present)
- **Gini close to 1** → node has a mix of many classes and is highly impure

Lower Gini = better (purer) split.

B. Splitting Criteria for Regression — RSS and MSE

Residual Sum of Squares (RSS): measures the total squared difference between actual and predicted values.

$$RSS = \sum(y - \hat{y})^2$$

Mean Squared Error (MSE): simply RSS divided by the number of samples.

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

Where:

- **y** = actual value
- **ŷ (y-hat)** = predicted value
- **n** = number of samples

Lower RSS or MSE = better split. CART picks the threshold that minimizes prediction error.

5. Pruning in CART

- Pruning trims branches of the tree that add little or no improvement to accuracy
- Purpose: prevent overfitting, simplify the tree, and improve generalization to new data

Types of Pruning

Type	How it Works
Cost Complexity Pruning	Compares the trade-off between tree accuracy and tree size — keeps only nodes that significantly improve performance
Reduced Error Pruning	Removes nodes that don't improve accuracy on a validation dataset — keeps only beneficial splits

Common Stopping / Pruning Criteria

- Maximum tree depth reached — tree stops growing after a predefined depth
- Minimum samples required to split — stops if a node has too few samples
- No further improvement in impurity — stops if a split doesn't reduce impurity/error
- Node becomes pure — stops when all samples in a node belong to one class

6. Important Hyperparameters in CART

Hyperparameter	What it Controls
max_depth	Maximum depth of the tree — prevents it from growing too deep and overfitting
min_samples_split	Minimum samples required to split an internal node — higher values = more conservative tree
min_samples_leaf	Minimum samples allowed in a leaf node — avoids very small/unstable leaves
max_features	Number of features considered when looking for the best split — reduces computation and variance
criterion	The function used to evaluate splits — Gini or Entropy for classification, MSE for regression

7.3 Why CART Is Popular

- Simplicity of results — trees can be read as a set of easily interpretable if-then rules, avoiding the need to compute scores with complex equations.
- Non-parametric and non-linear — CART makes no assumption that the predictors and target are linearly or otherwise functionally related, which suits messy real-world data.
- Implicit feature selection — attributes chosen near the top of the tree are automatically identified as the most informative for the prediction task.
- Foundation for ensembles — CART is the base learner used inside more powerful ensemble methods such as Bagging, Random Forests and Boosting (covered in Unit V).

Limitations

- Overfitting — an unpruned tree can memorise noise in the training data, hurting generalisation.
- High variance — a small change in the training data can lead to a very different tree being learned.
- Low bias for deep trees — very complex trees fit the training data extremely well but may generalise poorly to new data.

8. Error Bounds

Fitting a model to training data is not the end goal — what matters is how well the model performs on new, unseen data.

8.1 Training Error vs. Generalization (True) Error

Definition: Training error
The error (fraction of misclassifications, or average squared error for regression) that a model makes on the very data it was trained on.

Definition: Generalization error (true error)
The expected error of a model on new instances drawn from the same distribution as the training data, but not seen during training.

A model with low training error but high generalization error is said to overfit — it has effectively memorised the training set (including its noise) rather than learning the underlying pattern. A model with high error on both training and unseen data is said to underfit — it is too simple to capture the pattern at all.

8.2 The Bias–Variance View of Error

The generalization error of a model can be conceptually decomposed into three parts: bias (error from overly simplistic assumptions, e.g. a linear model fit to curved data), variance (error from sensitivity to the particular training sample, e.g. a very deep, unpruned decision tree), and irreducible noise (error that no model can eliminate, coming from randomness inherent in the data). Simpler models (linear regression, shallow/pruned trees) tend to have higher bias and lower variance; more flexible models (deep decision trees, high-degree polynomial regression) tend to have lower bias but higher variance. Choosing model complexity is therefore a trade-off, and is central to why techniques like tree pruning and regularisation exist.

8.3 A Formal Error Bound

Task type	Common error metric	What it measures
Classification	Error rate = 1 − Accuracy	Fraction of instances misclassified

Task type	Common error metric	What it measures
Regression	Mean Squared Error (MSE)	Average squared residual between prediction and actual value
Regression	R^2 / Adjusted R^2	Proportion of variance in the target explained by the model